Estimation Probabiliste de la Fin d'Alerte Orage

Modélisation par Processus de Hawkes et Gradient Boosting

Data Battle 2026 — Meteorage

Résumé

Ce rapport présente une approche en deux volets pour estimer la probabilité de fin d'une alerte foudre autour d'aéroports européens, à partir de données d'impacts fournis par Meteorage. Le premier volet, probabiliste, repose sur la modélisation des impacts comme un processus ponctuel auto-excitant de Hawkes, dont les paramètres sont estimés par maximum de vraisemblance, et dont la validité est évaluée par le théorème de rescaling temporel et par simulation Monte Carlo. À partir du modèle calibré, on calcule en temps réel un délai optimal de levée d'alerte u^* sous un risque toléré ε. Le second volet, supervisé, traduit le même problème en une tâche de régression sur des features temporelles construites à partir des historiques d'impacts, et utilise LightGBM pour apprendre les patterns de fin d'alerte directement depuis les données. Les deux approches sont complémentaires : la première fournit une interprétabilité théorique et une quantification rigoureuse de l'incertitude, la seconde offre une flexibilité et une puissance prédictive supérieures lorsque le volume de données le permet.

Table des matières

1 Vue d'ensemble des deux stratégies **3**

2 Formulation mathématique du problème **3**
2.1 Espace de probabilité et données . . . 3
2.2 Processus ponctuel marqué . . . 3
2.3 Définition formelle d'une alerte . . . 4
2.4 Historique et filtration . . . 4
2.5 Objectif probabiliste . . . 4

3 Stratégie 1 : Modélisation par processus de Hawkes **4**
3.1 Choix du modèle . . . 4
3.2 Lien avec la probabilité de survie . . . 5
3.3 Variable récursive et calcul en $O(n)$ . . . 6

4 Prétraitement des données — `preprocessing.py` **6**
4.1 Description . . . 6
4.2 Procédure . . . 6
4.3 Interface . . . 7

5 Analyse exploratoire — `eda.py` **7**
5.1 Description . . . 7
5.2 Quantités analysées . . . 7
5.3 Interface . . . 7

6 Estimation par maximum de vraisemblance — `hawkes.py` **8**
6.1 Log-vraisemblance d'Ogata . . . 8
6.2 Estimateur du maximum de vraisemblance . . . 8
6.3 Résultats obtenus . . . 8
6.4 Interface . . . 9

7 Validation du modèle — `validation.py` **9**
7.1 Théorème de rescaling temporel . . . 9
7.2 Calcul analytique des résidus . . . 9
7.3 Test statistique . . . 10
7.4 Validation par simulation . . . 10
7.5 Interface . . . 10

8 Simulation par algorithme de thinning — `simulation.py` **11**
8.1 Algorithme de thinning d'Ogata . . . 11
8.2 Interface . . . 11

9 Calcul du délai optimal u^* — `prediction.py` **11**
9.1 Estimation de la courbe de survie par Monte Carlo . . . 11
9.2 Interface . . . 12
9.3 Exemple de résultat attendu . . . 12

1 Vue d'ensemble des deux stratégies

Le tableau 1 résume les deux approches développées dans ce rapport et leur articulation.

	Approche 1 : Processus de Hawkes	**Approche 2 : LightGBM**
Paradigme	Modèle génératif probabiliste	Modèle discriminatif supervisé
Objectif direct	Estimer $\mathbb{P}(T > \tau_n + u \mid \mathcal{H}_n)$	Prédire u^* à partir de features
Entrées	Séquences temporelles $\{\tau_i\}$	Vecteur de features $\mathbf{f}_n$ construit depuis $\mathcal{H}_n$
Paramètres	$(\hat{\mu}, \hat{\alpha}, \hat{\beta})$ par MLE	Arbres de décision boostés, appris par CV
Validation	Théorème de rescaling + simulation Monte Carlo	Métriques de régression (MAE, RMSE) sur ensemble de test
Interprétabilité	Totale : chaque paramètre a un sens physique	Partielle : via SHAP values
Hypothèses	Stationnarité par aéroport, noyau exponentiel	Aucune hypothèse distributionnelle
Avantages	Quantification de l'incertitude, extrapolation	Capture les non-linéarités, les marques
Limites	Hypothèse paramétrique forte, $\sim$150 alertes	Boîte noire, pas de distribution de u^*

Table 1 – Comparaison des deux stratégies de modélisation.

Remarque 1.1. *Nous recommandons de développer les deux approches en parallèle. L'approche probabiliste est indispensable pour la rigueur mathématique, la quantification de l'incertitude et la compréhension du phénomène. LightGBM sera plus compétitif sur le score final car il peut intégrer naturellement les marques (amplitude, position, type d'éclair) sans hypothèse de forme sur leur rôle dans la dynamique.*

2 Formulation mathématique du problème

2.1 Espace de probabilité et données

On se place sur un espace de probabilité filtré $(\Omega, \mathcal{F}, (\mathcal{F}_t)_{t\geq 0}, \mathbb{P})$. Soit $A \in \mathbb{R}^2$ le centre géographique d'un aéroport et $r = 20$ km. On note :

$$\mathcal{D} = B(A, r) = \{x \in \mathbb{R}^2 \mid \|x - A\| \leq r\}$$

le disque de surveillance dans lequel les impacts déclenchent et prolongent les alertes.

2.2 Processus ponctuel marqué

Les impacts de foudre survenant dans $\mathcal{D}$ sont modélisés par un **processus ponctuel marqué** :

$$\mathcal{N} = \{(\tau_i, \mathbf{X}_i)\}_{i\geq 1}$$

où, pour chaque impact i :

— $\tau_i \in \mathbb{R}_+$ est l'**instant d'impact** (colonne `date`),
— $\mathbf{X}_i = (\xi_i,\, \phi_i,\, \alpha_i,\, d_i,\, C_i) \in \mathcal{X}$ est le **vecteur de marques** composé de :
  — $(\xi_i, \phi_i) \in \mathcal{D}$: position géographique (`lon`, `lat`),
  — $\alpha_i \in \mathbb{R}$: amplitude du signal en kA (`amplitude`),
  — $d_i \in [0, r]$: distance à l'aéroport (`dist`),
  — $C_i \in \{0, 1\}$: type d'éclair, nuage-sol ou intra-nuageux (`icloud`).

2.3 Définition formelle d'une alerte

Définition 2.1 (Alerte)**.** *Une **alerte** est déclenchée à l'instant τ_0 dès qu'un impact survient dans $\mathcal{D}$. Elle se termine à la première période de $\Delta = 30$ minutes consécutives sans impact dans $\mathcal{D}$. La **durée totale de l'alerte** est la variable aléatoire :*

$$T = \inf\{t > \tau_0 \mid \mathcal{N}((t,\, t+\Delta) \cap \mathcal{D} \times \mathcal{X}) = 0\} + \Delta - \tau_0.$$

2.4 Historique et filtration

On se place à l'instant courant τ_n, instant du n-ième impact dans $\mathcal{D}$, avec :

$$\tau_0 < \tau_1 < \cdots < \tau_n, \qquad \tau_i - \tau_{i-1} < \Delta \quad \forall\, i \in \{1, \ldots, n\}.$$

On définit la **filtration naturelle** du processus et l'**historique observé** jusqu'à τ_n :

$$\mathcal{H}_n = \sigma(\tau_i,\, \xi_i, \phi_i, \alpha_i, d_i, C_i \mid 0 \le i \le n)\,.$$

C'est ce vecteur aléatoire observé qui constitue la statistique exhaustive permettant toute inférence sur la dynamique future du processus.

2.5 Objectif probabiliste

Définition 2.2 (Délai optimal de levée d'alerte)**.** *Soit $\varepsilon \in (0, 1)$ le **niveau de risque toléré** par l'aéroport. À chaque nouvel impact en τ_n, on cherche le **délai minimal de levée d'alerte** $u^* \in [0, \Delta]$ tel que :*

$$\boxed{u^* = \inf\{u \in [0, \Delta] \mid \mathbb{P}(T > \tau_n + u \mid \mathcal{H}_n) < \varepsilon\}\,.}$$

Interprétation. Après u^* minutes sans nouvel éclair depuis τ_n, on peut admettre sous risque ε que l'alerte est terminée, permettant à l'aéroport de reprendre son activité avant l'expiration du délai réglementaire fixe de $\Delta = 30$ min.

3 Stratégie 1 : Modélisation par processus de Hawkes

3.1 Choix du modèle

Pour modéliser la dynamique temporelle des impacts dans $\mathcal{D}$, nous choisissons un **processus de Hawkes univarié à noyau exponentiel**. Ce choix est motivé par la propriété d'auto-excitation : un impact augmente la probabilité d'impacts proches dans le temps, ce qui est physiquement cohérent avec la dynamique des orages où un éclair favorise la création de nouveaux éclairs.

Définition 3.1 (Processus de Hawkes à noyau exponentiel)**.** *L'intensité conditionnelle du processus est définie par :*

$$\lambda^*(t) = \mu + \alpha \sum_{\tau_i < t} e^{-\beta(t - \tau_i)},$$

où :

— $\mu > 0$ *est le* ***taux de base*** *(activité orageuse de fond),*

— $\alpha > 0$ *est le paramètre d'****excitabilité*** *(amplitude du saut à chaque impact),*

— $\beta > 0$ *est le* ***taux de décroissance*** *(vitesse d'atténuation de l'excitation).*

Remarque 3.1. *La condition de stationnarité du processus exige $\alpha < \beta$, soit $\alpha/\beta < 1$. Ce ratio mesure la* ***branchabilité*** *du processus : un ratio proche de 1 indique un processus proche de la criticité, où chaque impact génère presque autant d'impacts secondaires qu'il n'en reçoit. Les résultats obtenus (ratio ≈ 0.98 pour tous les aéroports) indiquent des orages très auto-excités, cohérents avec la physique orageuse.*

3.2 Lien avec la probabilité de survie

Pour établir l'expression de $\mathbb{P}(T > \tau_n + u \mid \mathcal{H}_n)$, on procède en deux étapes : d'abord sur un processus de Poisson *déterministe* pour faire apparaître l'exponentielle, puis on revient au cas du Hawkes où l'intensité est *aléatoire* pour faire apparaître l'espérance.

Étape 1 — Processus de Poisson inhomogène à intensité déterministe

Supposons provisoirement que $\lambda^*(s) = \lambda(s)$ est une fonction **déterministe** du temps (non aléatoire). Le nombre d'impacts $N([a, b])$ dans un intervalle $[a, b]$ suit alors une loi de Poisson de paramètre :

$$\Lambda(a, b) = \int_a^b \lambda(s)\, ds.$$

La probabilité qu'il y en ait **zéro** est donnée par la formule de la loi de Poisson pour $k = 0$:

$$\mathbb{P}\big(N([a, b]) = 0\big) = \frac{e^{-\Lambda(a,b)}\Lambda(a, b)^0}{0!} = e^{-\Lambda(a,b)} = \exp\left(-\int_a^b \lambda(s)\, ds\right).$$

C'est de là que vient l'exponentielle. On note en particulier que $\Lambda(a, b)$ est un *nombre d'impacts attendus*, pas une probabilité : il peut dépasser 1, mais $e^{-\Lambda}$ reste bien dans $[0, 1]$.

Étape 2 — Retour au processus de Hawkes : l'intensité est aléatoire pour $s > \tau_n$

Dans le processus de Hawkes, l'intensité conditionnelle est :

$$\lambda^*(s) = \mu + \alpha \sum_{\tau_i < s} e^{-\beta(s-\tau_i)}.$$

À l'instant courant τ_n, la somme porte sur tous les impacts $\tau_i < s$. Or pour $s > \tau_n$, cette somme inclut les impacts $\tau_{n+1}, \tau_{n+2}, \ldots$ qui **pourraient survenir dans** (τ_n, s) et que l'on ne connaît pas encore. Ainsi, pour $s > \tau_n$:

$\lambda^*(s)$ est une **variable aléatoire**, dépendant de la trajectoire future du processus.

Par conséquent, $\int_{\tau_n}^{\tau_n+u} \lambda^*(s)\, ds$ est elle-même aléatoire.

Étape 3 — Mise en espérance par conditionnement total

On note $\mathcal{F}$ la tribu engendrée par la trajectoire complète du processus (passée *et* future). Par la formule des probabilités totales, en conditionnant d'abord à $\mathcal{F}$ puis en intégrant sur les trajectoires futures :

$$\mathbb{P}(T > \tau_n + u \mid \mathcal{H}_n) = \mathbb{E}\Big[\mathbb{P}\big(T > \tau_n + u \mid \mathcal{F}\big) \;\Big|\; \mathcal{H}_n\Big].$$

Conditionnellement à $\mathcal{F}$, la trajectoire future est *entièrement fixée*, donc $\lambda^*(s)$ est déterministe sur chaque trajectoire ω et l'étape 1 s'applique :

$$\mathbb{P}\big(T > \tau_n + u \mid \mathcal{F}\big)(\omega) = \exp\left(-\int_{\tau_n}^{\tau_n+u} \lambda^*_\omega(s)\, ds\right).$$

En substituant dans la formule des probabilités totales :

$$\boxed{\mathbb{P}(T > \tau_n + u \mid \mathcal{H}_n) = \mathbb{E}\left[\exp\left(-\int_{\tau_n}^{\tau_n+u} \lambda^*(s)\, ds\right) \middle| \mathcal{H}_n\right].}$$

Pourquoi cette espérance n'a pas d'expression analytique

Si $\lambda^*(s)$ était déterministe (Poisson inhomogène pur), le calcul serait terminé. Dans le Hawkes, $\lambda^*(s)$ dépend de ses propres réalisations futures : chaque nouvel impact augmente λ^*, ce qui rend probable un impact suivant, qui à son tour augmente λ^*, etc. Ce **couplage auto-excitant** entre l'intensité et les événements qu'elle génère crée une dépendance récursive qui empêche le calcul en forme fermée.

On estime donc cette espérance par **simulation Monte Carlo** : on simule N trajectoires futures $\omega^{(1)}, \ldots, \omega^{(N)}$ depuis τ_n à l'aide de l'algorithme de thinning, et on estime :

$$\mathbb{P}(T > \tau_n + u \mid \mathcal{H}_n) \;\approx\; \frac{1}{N}\sum_{j=1}^{N} \mathbf{1}\Big[\text{aucun impact dans } (\tau_n,\, \tau_n + u) \text{ dans la simulation } \omega^{(j)}\Big],$$

ce qui revient à compter la proportion de simulations où l'alerte est encore active u minutes après le dernier impact observé.

3.3 Variable récursive et calcul en $O(n)$

Un point technique central est l'introduction de la variable récursive :

$$R_i = \sum_{j=0}^{i-1} e^{-\beta(\tau_i - \tau_j)}, \quad R_0 = 0,$$

qui se met à jour en $O(1)$ à chaque nouveau pas de temps :

$$R_i = e^{-\beta(\tau_i - \tau_{i-1})}\,(1 + R_{i-1}).$$

L'intensité conditionnelle s'écrit alors $\lambda^*(\tau_i) = \mu + \alpha R_i$, et toutes les sommes sur l'historique se calculent sans boucle imbriquée.

4 Prétraitement des données — `preprocessing.py`

4.1 Description

Le module `preprocessing.py` lit le fichier CSV brut et construit la structure de données fondamentale : les **trajectoires d'alertes**, unités d'observation de l'inférence statistique.

4.2 Procédure

1. Chargement du CSV et parsing des dates en timestamps UTC.
2. Filtrage sur `dist` ≤ 20 km : seuls ces impacts appartiennent à $\mathcal{D}$.
3. Pour chaque aéroport, tri chronologique et **segmentation en alertes** : dès qu'un silence de $\Delta = 30$ min est détecté entre deux impacts consécutifs, l'alerte courante se termine et une nouvelle débute au prochain impact.
4. Exclusion des alertes à un seul impact (pas d'inter-arrivée exploitable pour la log-vraisemblance).

4.3 Interface

```python
def load_data(path: str) -> tuple[pd.DataFrame, pd.DataFrame, dict]:
    """
    Entrees :
        path : str  -- chemin vers le CSV brut (507 071 lignes)

    Sorties :
        df       : DataFrame  -- donnees brutes completes
        df_inner : DataFrame  -- impacts filtres dist <= 20 km
        alerts   : dict       -- {airport: [traj_1, ..., traj_K]}
                                 traj_k = array numpy de timestamps
                                 en secondes relatifs a tau_0=0
    """
```

Structure de la sortie. Le dictionnaire `alerts` a la forme :

$$\texttt{alerts}[\text{airport}] = \left[\underbrace{[0,\, \tau_1^{(1)}, \ldots, \tau_{n_1}^{(1)}]}_{\text{alerte } 1},\ \ldots,\ \underbrace{[0,\, \tau_1^{(K)}, \ldots, \tau_{n_K}^{(K)}]}_{\text{alerte } K}\right].$$

Chaque trajectoire ω_k est un array numpy de timestamps en secondes depuis $\tau_0^{(k)} = 0$. Le tableau 2 donne la répartition des alertes.

Aéroport	Alertes	Impacts dans $\mathcal{D}$
Ajaccio	34	∼800
Bastia	30	∼700
Biarritz	25	∼600
Nantes	38	∼900
Pise	25	∼600
Total	**152**	**∼3600**

Table 2 – Répartition des alertes par aéroport après prétraitement.

5 Analyse exploratoire — `eda.py`

5.1 Description

Le module `eda.py` calcule et visualise les distributions empiriques des trois quantités clés pour justifier le choix du modèle de Hawkes et identifier les différences géographiques.

5.2 Quantités analysées

1. **Inter-arrivées** $\delta_i = \tau_i - \tau_{i-1}$ (en minutes) : si elles décroissent exponentiellement, cela est cohérent avec un processus de Poisson inhomogène et compatible avec le Hawkes.
2. **Durées d'alertes** $T_k = \tau_{n_k}^{(k)} + \Delta$ (en minutes) : donne la distribution de la variable que l'on cherche à prédire.
3. **Nombre d'impacts par alerte** n_k : révèle l'intensité typique des orages par géographie.

5.3 Interface

```python
def eda(alerts: dict) -> None:
    """
    Entrees :
        alerts : dict  -- sortie de load_data()

    Sorties :
        Figures sauvegardees dans figures/eda_*.png
        (histogrammes superposes par aeroport)
    """
```

6 Estimation par maximum de vraisemblance — `hawkes.py`

6.1 Log-vraisemblance d'Ogata

Pour une trajectoire $\omega_k = \{\tau_0, \tau_1, \ldots, \tau_n\}$ observée sur $[0, T_k]$ avec $T_k = \tau_n + \Delta$, la log-vraisemblance du processus de Hawkes est donnée par le théorème de Ogata (1988) :

$$\ell_k(\mu, \alpha, \beta) = \underbrace{\sum_{i=0}^{n} \log \lambda^*(\tau_i)}_{\text{contribution de chaque impact}} - \underbrace{\int_0^{T_k} \lambda^*(s)\, ds}_{\text{terme de compensation}} .$$

Le terme intégral se calcule analytiquement :

$$\int_0^{T_k} \lambda^*(s)\, ds = \mu T_k + \frac{\alpha}{\beta} \sum_{i=0}^{n} \left(1 - e^{-\beta(T_k - \tau_i)}\right).$$

Sur l'ensemble des K alertes d'un aéroport, on maximise la log-vraisemblance totale :

$$\mathcal{L}(\mu, \alpha, \beta) = \sum_{k=1}^{K} \ell_k(\mu, \alpha, \beta).$$

6.2 Estimateur du maximum de vraisemblance

Définition 6.1 (EMV du Hawkes)**.** *L'estimateur du maximum de vraisemblance est :*

$$(\hat{\mu}, \hat{\alpha}, \hat{\beta}) = \arg \max_{\mu>0,\, \alpha>0,\, \beta>0} \mathcal{L}(\mu, \alpha, \beta).$$

La maximisation est réalisée numériquement par l'algorithme L-BFGS-B avec une grille de points d'initialisation pour éviter les minima locaux.

6.3 Résultats obtenus

Le tableau 3 présente les paramètres estimés pour chaque aéroport.

Aéroport	$\hat{\mu}$	$\hat{\alpha}$	$\hat{\beta}$	$\hat{\alpha}/\hat{\beta}$
Ajaccio	0.00376	4.235	4.335	0.977
Bastia	0.00363	4.966	5.031	0.987
Biarritz	0.00450	5.524	5.612	0.984
Nantes	0.00240	5.289	5.379	0.983
Pise	0.00591	4.681	4.762	0.983

Table 3 – Paramètres du processus de Hawkes estimés par MLE pour chaque aéroport.

Remarque 6.1. *Le ratio $\hat{\alpha}/\hat{\beta}$ proche de 1 pour tous les aéroports indique des processus proches de la criticité : chaque impact génère en moyenne $\hat{\alpha}/\hat{\beta} \approx 0.98$ impacts secondaires. Pise présente le taux de base $\hat{\mu}$ le plus élevé (0.00591), cohérent avec une activité orageuse de fond plus forte en Toscane.*

6.4 Interface

```python
def neg_log_likelihood(params: list, trajectories: list) -> float:
    """
    Entrees :
        params       : [mu, alpha, beta]  -- parametres courants
        trajectories : list of np.array   -- trajectoires d'une meme ville

    Sortie :
        float  -- valeur de -L(mu, alpha, beta) a minimiser
    """

def fit_hawkes(trajectories: list) -> dict:
    """
    Entrees :
        trajectories : list of np.array  -- alertes d'un aeroport

    Sortie :
        dict avec cles 'mu', 'alpha', 'beta', 'result' (objet scipy)
    """

def fit_all_airports(alerts: dict) -> dict:
    """
    Entrees :
        alerts : dict  -- sortie de load_data()

    Sortie :
        dict {airport: params_dict}  -- un modele calibre par aeroport
    """
```

7 Validation du modèle — `validation.py`

7.1 Théorème de rescaling temporel

La validation repose sur le résultat fondamental suivant.

Théorème 7.1 (Papangelou, 1972 — Rescaling temporel)**.** *Soit $\{\tau_i\}$ une réalisation d'un processus ponctuel d'intensité conditionnelle $\lambda^*(t)$. On définit la* ***compensatrice*** *:*

$$\Lambda(\tau_i) = \int_0^{\tau_i} \lambda^*(s)\, ds.$$

Si le modèle est correctement spécifié, les accroissements :

$$U_i = \Lambda(\tau_i) - \Lambda(\tau_{i-1}), \quad i = 1, 2, \ldots$$

sont des variables aléatoires indépendantes et identiquement distribuées de loi $\mathrm{Exp}(1)$.

7.2 Calcul analytique des résidus

Pour le noyau exponentiel, l'accroissement de la compensatrice entre τ_{i-1} et τ_i se calcule analytiquement :

$$U_i = \mu(\tau_i - \tau_{i-1}) + \frac{\alpha R_{i-1}}{\beta}\left(1 - e^{-\beta(\tau_i - \tau_{i-1})}\right),$$

où $R_{i-1} = \sum_{j<i-1} e^{-\beta(\tau_{i-1}-\tau_j)}$ est la variable récursive évaluée en τ_{i-1}.

7.3 Test statistique

On applique un **test de Kolmogorov-Smirnov** sur l'ensemble des résidus $\{U_i\}$ contre la loi Exp(1), complété d'un QQ-plot pour le diagnostic visuel.

Résultat obtenu. Le test est rejeté à 5% pour Ajaccio (KS $= 0.0798$, $p < 0.001$). Ce rejet n'invalide pas l'approche : il indique que le modèle homogène de base peut être amélioré, notamment en intégrant les marques (amplitude, distance) dans l'intensité conditionnelle, ou en ajoutant une composante saisonnière dans μ.

7.4 Validation par simulation

La seconde approche de validation, décrite par le module `simulation_validation()`, consiste à :

1. Simuler $N_{\text{sim}} = 500$ alertes avec les paramètres $(\hat{\mu}, \hat{\alpha}, \hat{\beta})$ estimés,
2. Comparer les distributions simulées (durées, nombre d'impacts, inter-arrivées) aux distributions observées par superposition des histogrammes.

Si les distributions se superposent, le modèle capture la dynamique essentielle des données.

7.5 Interface

```python
def compute_residuals(params: dict, trajectories: list) -> np.ndarray:
    """
    Entrees :
        params       : dict          -- {mu, alpha, beta}
        trajectories : list          -- alertes d'un aeroport

    Sortie :
        np.ndarray  -- array des residus
                       U_i = Lambda(tau_i) - Lambda(tau_{i-1})
                       doit suivre Exp(1) si le modele est correct
    """

def goodness_of_fit(params: dict, trajectories: list, airport: str):
    """
    Entrees :
        params, trajectories, airport  -- idem ci-dessus

    Sorties :
        (stat, pvalue)  -- statistique KS et p-value
        Figures : QQ-plot et histogramme vs Exp(1) dans figures/
    """

def simulation_validation(params: dict, trajectories: list,
                          airport: str, n_sim: int = 500):
    """
    Entrees :
        n_sim : int  -- nombre d'alertes simulees (defaut 500)

    Sorties :
        Figure comparant distributions observees vs simulees
    """
```

8 Simulation par algorithme de thinning — `simulation.py`

8.1 Algorithme de thinning d'Ogata

La simulation exacte du processus de Hawkes repose sur l'algorithme de thinning d'Ogata (1981). La difficulté est que l'intensité $\lambda^*(t)$ varie dans le temps, empêchant la simulation directe par inversion.

Principe. On construit une borne supérieure locale $\bar{\lambda} \geq \lambda^*(t)$ sur un intervalle $[t, t+\delta]$. On génère un candidat $t' = t + E$ où $E \sim \mathrm{Exp}(\bar{\lambda})$. On accepte t' comme vrai événement avec probabilité $\lambda^*(t')/\bar{\lambda}$, sinon on le rejette (c'est le « thinning » des candidats fantômes).

Pour le noyau exponentiel, l'intensité est décroissante entre deux événements, donc $\bar{\lambda} = \lambda^*(t^+)$ (intensité juste après le dernier événement) est une borne valide. La mise à jour récursive :

$$R_{\text{new}} = e^{-\beta \cdot dt}(1 + R_{\text{old}})$$

assure un coût $O(1)$ par candidat.

8.2 Interface

```python
def simulate_alert(mu: float, alpha: float, beta: float,
                   rng=None) -> np.ndarray:
    """
    Simule une alerte complete depuis tau_0 = 0
    jusqu'a la terminaison naturelle (silence de 30 min).

    Entrees :
        mu, alpha, beta : float  -- parametres du processus
        rng : np.random.Generator (optionnel)

    Sortie :
        np.ndarray  -- timestamps en secondes depuis tau_0 = 0
    """

def simulate_hawkes(mu: float, alpha: float, beta: float,
                    T_max: float, history: list = None,
                    rng=None) -> list:
    """
    Simule le processus sur [0, T_max], eventuellement
    conditionne a un historique passe (history).

    Entrees :
        T_max   : float  -- horizon de simulation (secondes)
        history : list   -- evenements passes (timestamps)

    Sortie :
        list  -- nouveaux evenements simules dans [0, T_max]
    """
```

9 Calcul du délai optimal u^* — `prediction.py`

9.1 Estimation de la courbe de survie par Monte Carlo

L'estimation de $\mathbb{P}(T > \tau_n + u \mid \mathcal{H}_n)$ est réalisée par simulation Monte Carlo.

Initialisation. On initialise R depuis l'historique observé :

$$R = \sum_{i=0}^{n} e^{-\beta(\tau_n - \tau_i)}.$$

Simulation. On lance N_{sim} réalisations de la continuation du processus depuis τ_n, en enregistrant pour chaque simulation le délai $u^{(j)}$ jusqu'à la terminaison de l'alerte.

Estimation. La courbe de survie est estimée par :

$$\hat{S}(u) = \mathbb{P}(T > \tau_n + u \mid \mathcal{H}_n) \approx \frac{1}{N_{\text{sim}}} \sum_{j=1}^{N_{\text{sim}}} \mathbf{1}\left[u^{(j)} > u\right].$$

Calcul de u^*. On retourne le premier u de la grille $[0, \Delta]$ tel que $\hat{S}(u) < \varepsilon$.

9.2 Interface

```python
def estimate_survival_and_ustar(
        params: dict,
        history: np.ndarray,
        epsilon: float = 0.10,
        n_sim: int = 3000,
        rng=None) -> dict:
    """
    Entrees :
        params  : dict       -- {mu, alpha, beta} de l'aeroport
        history : np.ndarray -- timestamps en secondes depuis tau_0,
                                dernier element = tau_n (instant courant)
        epsilon : float      -- risque tolere (ex. 0.10 pour 10%)
        n_sim   : int        -- nombre de simulations Monte Carlo

    Sorties :
        dict avec :
            'u_star'   : float      -- delai optimal en secondes
            'u_grid'   : np.ndarray -- grille de u
                                       (300 points dans [0, Delta])
            'survival' : np.ndarray -- S(u) estimee sur u_grid
        Figure : courbe de survie avec u* et epsilon dans figures/
    """
```

9.3 Exemple de résultat attendu

Pour une alerte en cours avec $n = 5$ impacts observés et $\varepsilon = 10\%$, le résultat typique est de la forme :

$$u^* \approx 18 \text{ min} \quad \Longrightarrow \quad \text{Gain de } 30 - 18 = 12 \text{ min sur l'alerte réglementaire.}$$